My Courses

WY LAB2-1 决策树

请下载附件（包含 `data_eg.in` , `data_test.in` , `data_train.in` ）

c7790857412c812f3017727076016b559ebecd7d.zip (attachment/c779/c7790857412c812f3017727076016b559ebecd7d.zip)。

之前 `data_eg.in` 最后一行的标签“1” “2”写成了“0” “1”， 新的附件中已经更新，可以重新下载附件。

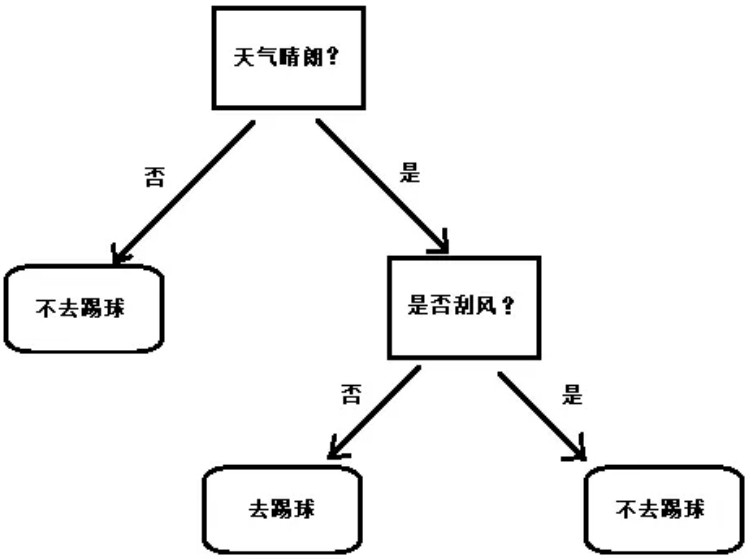
0. 背景知识

决策树是机器学习当中一个基础的模型，它的整体结构存储了一组**属性到值（类别）之间的映射关系**。一棵决策树的每个非叶子节点均表示一个属性上的测试，每个分支则代表一个测试输出，每个叶节点代表一种决策结果。每个节点包含的样本集合根据属性测试的结果被划分到相应的子节点中，该节点的子节点数量，也就是测试结果的分支数量，由该属性的值与决策树的设计方式决定，如常见的“是否拥有生命？”等仅拥有二值的属性测试，就往往拥有2个分支，一个代表答案为“是”，一个代表答案为“否”。当拥有完整的一棵决策树时，对一组属性作判定的流程为：

1. 从根节点出发。

2. 如果为叶节点，则返回该节点上的决策结果。

3. 进行该节点上的属性测试，并根据结果进入对应的分支，到达其中一个子节点后，回到步骤2。



进行决策树的测试需要拥有两大要素。一是决策树本身，二是用于测试的实例的属性。图中的决策树对应的实例的属性有两个：一为天气是否晴朗、二为是否刮风。

1. 理解决策树的构建

1.1 基本原理

在实际应用当中，决策树的构建是至关重要的。在已知任务的前提下构建一棵决策树，可以依赖于个人或专家的经验，但在更庞大的属性集合、更抽象无序的任务中，我们需要学习由数据驱动构建决策树的办法。

通过一系列格式为“属性-对应的决策结果”的数据获取决策树的过程本质上是一种监督学习，所谓监督学习就是给定一堆样本，每个样本都有一组属性和一个类别，这些类别是事先确定的，那么通过学习得到一个分类器，这个分类器能够对新出现的对象给出正确的分类。这样的机器学习就被称之为监督学习。

构建决策树的框架主要为一个自上而下的分治：

1. 从根出发（对应全部的数据）。

2. 如果当前节点满足终止条件，则该节点为叶节点，确认决策结果，并返回。

3. 否则，需要为划分余下的数据选择一个属性测试，将它们继续往下划分。划分完以后，不同的数据根据属性测试的结果划分入不同的子节点，子节点再分别进入步骤2。

终止条件主要有：

1. 如果当前节点包含的所有数据都对应同一个分类（即决策结果），则无需再往下划分。

2. 如果当前节点包含的所有数据属性都相同，则无法划分，依据一定的规则确定决策结果后返回，通常的选择为在数据中出现次数最多的分类。

3. 如果当前节点不包含任何数据，则任意设置一个决策结果并返回。

因此，决策算法的**关键**在于如何确定用于划分的属性。

1.2 属性的选择

如何为给定的数据集选择用于划分的属性，是个十分复杂的问题。它具有不同的评估标准，因此也产生了许多种不同的决策树算法。总地来说，我们希望节点所包含的数据尽可能属于同一类别，即节点的“纯度”尽可能高，这样可以更高效地得到决策结果。

信息熵就是这样一种衡量数据“纯度”的指标。假定当前数据当中第k类（拥有第 k 种决策结果）的数据所占比例为p_k，那么该数据集 D 的信息熵定义为

$$Ent(D) = -\sum p_k \log_2 p_k$$

$Ent(D)$ 的值越小，代表着数据集 D 的“纯度”越高。

1.2.1 ID3决策树算法

ID3决策树算法直接基于信息熵的概念。其思路为选择划分后能够使得信息熵变低的幅度尽可能大的属性。假定我们使用属性 A 对数据集 D 进行划分以后，D 被划分为 V 个子集 $\{D_1, D_2, ..., D_V\}$。定义用属性 A 划分 D 带来的**信息增益**为

$$Gain(D, A) = Ent(D) - \sum_{v=1}^{V} \frac{|D_v|}{|D|} Ent(D_v)$$

信息增益越大，则意味着使用属性 A 来进行划分所获得的“纯度提升”越大。使用该准则选择每一个非叶节点上用于划分的属性，就是ID3决策树算法。

1.2.2 CART决策树算法

另有一衡量数据“纯度”的指标叫做**基尼系数**（Gini Index），它反映了从一组数据当中随机抽取两个样本，他们所属类别不同的概率。同样假定当前数据当中第 k 类（拥有第 k 种决策结果）的数据所占比例为p_k，数据集 D 的基尼系数定义为

$$G(D) = \sum_k \sum_{k' \neq k} p_k p_{k'} = 1 - \sum_k p_k^2$$

$G(D)$ 的值越小，代表着数据集 D 的“纯度”越高。

假定我们使用属性 A 对数据集 D 进行划分以后，D 被划分为 V 个子集 $\{D_1, D_2, ..., D_V\}$。定义用属性 A 划分 D 后的基尼系数为

$$Gini(D, A) = \sum_{v=1}^{V} \frac{|D_v|}{|D|} G(D_v)$$

CART算法的选择策略是：在候选属性集合中，选取那个使划分后基尼指数**最小**的属性。

1.2.3 练习

数据编号	根蒂	敲声	纹理	判断
1	蜷缩	沉闷	清晰	好瓜
2	硬挺	清脆	清晰	坏瓜
3	蜷缩	沉闷	模糊	坏瓜
4	硬挺	清脆	模糊	坏瓜
5	蜷缩	清脆	清晰	坏瓜
6	蜷缩	沉闷	模糊	好瓜

上为一组西瓜属性与好坏程度的数据集，它由 6 个样本构成。其中每个样本都是一个西瓜，具有根蒂形态、敲声、纹理三种属性。最后一列为样本的类别，共两类，分别为好瓜、坏瓜。

你需要为此数据集使用上文提到的 ID3 与 CART 算法**分别**构建出决策树。为了检验你对决策树的构建流程与这两个算法的理解程度，**你需要写出计算的过程，并展示决策树最终的形态。**

2. 实现不同的决策树算法

在上一节，你已经掌握了两种决策树算法完整的计算过程。现在，为了应对更庞大的数据，是时候用代码来实现这两种决策树的构建了。为了简化实现的细节，你需要构造的是一棵**二叉决策树**。也就是说，你只需要考虑处理的数据的每个属性的取值只有 2 的情况。

你需要自行实现一份名为 `decisionTree.cpp` 的代码，需要满足

- 能够读入数据集，并建出该数据集生成的决策树。
- 数据集的格式为
  - 第一行两个数 n, m 。n 表示数据的条数，m 表示每一个数据具有的属性数目。
  - 接下来 n 行，每行 $m+1$ 个数，表示一个数据。前 m 个数表示该数据 m 个属性的取值，每一项都只会出现 $0, 1$ 两种取值。 最后一个数为该条数据的类别，为一个 $[1, n]$ 范围内的正整数。
  - 为了进一步理解和检验数据集的格式，你可以查看下发文件中的 `data_eg.in` 。该文件是上文小练习中数据集转换到这个格式下的版本。
- 存储决策树的方式不限。
- 在 `build_ID3()` 函数内完成使用 ID3 算法的决策树构建的主体过程。
- 在 `build_CART()` 函数内完成使用 CART 算法的决策树构建的主体过程。
- 在 `decisionTree.cpp` 文件当中适当地加一些注释说明函数的功能，并在实验报告中简述实现的思路。

(要求使用C++代码实现，但可以使用STL标准库)

进行测试

决策树的构建生成了一套从属性到类别的映射，它是为了预测更多其他数据的类别而存在的。用于构造决策树的数据集具有完整的属性和类别信息，我们称其为**训练集**。当决策树构建完成后，为了衡量其**泛化能力**，就需要在一组之前未出现过的数据集上做评估。即，拿这些未出现数据的属性在决策树上做判定，将得到的决策结果与数据真实的类别作对比，得到准确率。这个用于评估决策树模型的数据集就叫做**测试集**。

你需要为训练集 `data_train.in` 构造决策树。 `data_train.in` 满足上一节当中描述的文件格式。

你需要在 `decisionTree.cpp` 中额外实现背景介绍当中对一组**属性作判定的流程**。为测试集 `data_test.in` 当中的数据统计准确率。 `data_test.in` 同样满足上一节当中描述的文件格式。在属性测试时，对于每一个样本，你只能使用它的 m 个属性值。在最后比较结果、统计准确率时，才能使用其所属类别。

假定 `data_test.in` 中共有 n_t 组数据。你需要将决策树对其属性决策的结果输出，格式为 n_t 行每行一个结果。你需要将使用 ID3 算法的结果输出到 `result_ID3.out` 中，将使用 CART 算法的结果输出到 `result_CART.out` 中。

在实验报告中汇报 ID3 与 CART 算法通过 `data_train.in` 构造的决策树在 `data_test.in` 测试集上的表现（统计准确率即可）。

思考题

1. 当我们要为一个数据集构建**二叉**决策树，而数据中有的属性是多离散值（例如：职业）时，我们应该如何处理？ 2. 当我们要为一个数据集构建**二叉**决策树，而数据中有的属性是连续值（例如：气温）时，我们应该如何处理？ 3. 对于上述数据当中有的属性是多离散值（例如：职业）的情况，构造二叉决策树与允许使用多叉决策树比，有何优劣？ 4. 对于数据中所有的属性都分别为二值、多离散值和连续值的情况，你能分别给出一个生活中可以使用决策树来帮助解决的例子吗？

最终提交

本题没有黑盒测试 不需要提交代码或标记最终版本，但需要签署Honor Code。下面的文件提交到“report”处。如果多次提交，将以最后一次提交的版本为准。

将下列文件置于同一个目录下打包，提交压缩文件：

1. `decisionTree.cpp` 源文件，需要包含适当的注释。包含第二节当中决策树的代码和第三节当中测试部分的代码。(30%)
2. `result_ID3.out` 和 `result_CART.out` . (10%)
3. 实验报告，格式不限。(60%)
    - 1.2.3 节中小练习的回答，需要包含计算过程和结果展示。(20%)
    - `decisionTree.cpp` 实现思路简述。(10%)
    - 两种算法的决策树在测试集 `data_test.in` 上的表现。(10%)
    - 思考题的回答。(20%)
4. 在完成题目中遇到的问题和解决方法。（不是必做项，不影响得分）
5. 对这道题目的感受和建议。（不是必做项，不影响得分）

UI powered by Twitter Bootstrap (http://getbootstrap.com/).
Tsinghua Online Judge is designed and coded by Li Ruizhe.
For all suggestions and bug reports, contact oj[at]liruizhe[dot]org.